



ANÁLISIS DE ALGORITMOS

Cifrado Cesar

FUERZA BRUTA VS. DIVIDE Y VENCERÁS

Autores:

Aurelio Rendón Viciego

Humberto de Jesús Peña D.

Ricardo Alejandro Rosas A.

Materia: Análisis de Algoritmos

CUCEI U. de G.

Fecha: 26 de Octubre de 2025

Índice

INTRODUCCIÓN	3	
El Problema del Descifrado César.		
	4	ENFOQUE 1
		Análisis de Fuerza Bruta.
ENFOQUE 2	5	
Adaptación a Divide y Vencerás (D&V).		
	7	RESULTADOS
		Comparativa de Rendimiento (Tiempo y Memoria).
CONCLUSIONES	9	
Lecciones y próximos pasos.		

¿Cuál es el Problema Central?



Objetivo: Encontrar un algoritmo más rápido y con menor uso de memoria que la simple iteración.

CIFRADO CÉSAR

Método simple de sustitución de letras por un "salto" constante.

EL RETO

Descifrar el texto sin conocer la clave (el valor del salto).

PROBLEMA DE LA FUERZA BRUTA

Probar las 25 posibles claves es lento y computacionalmente ineficiente.

Solución 1: Fuerza Bruta (Brute Force)

LÓGICA DIRECTA

Generar y almacenar las 25 posibles versiones del texto cifrado.

COMPLEJIDAD TEMPORAL

$O(n \cdot k)$, donde n es el tamaño del texto y k es el alfabeto (25).

DESVENTAJA (MEMORIA)

Alto costo de memoria al guardar 25 copias completas del texto descifrado.

```
// Lógica de Fuerza Bruta (cifradoCesar)
for i in range(1, 26): // Probar 25 claves
    texto_descifrado = []
    for letra in cifrada: // Recorrer el texto ENTERO
        // ... descifrar char ...
    resultados.append("".join(texto_descifrado))
// Almacenar 25 versiones
```


Solución 2: Divide y Vencerás (D&V)

Aplicando D&V al descifrado, evitamos la repetición innecesaria del trabajo. **$O(n)$**



DIVIDIR

Se usa una "palabra guía" conocida (o probable) del texto original.



BUSCAR

Se buscan candidatas (misma longitud) en el texto cifrado.



VENCER

Probar saltos solo en la palabra candidata para encontrar la clave única.

Resultado: Descubrir el salto correcto con una fracción del esfuerzo total de descifrado.

Lógica Clave de Divide y Vencerás

01

EVITAR DUPLICACIÓN

El diseño se enfoca en no generar ni almacenar 25 textos completos.

02

CALCULAR SALTO PROBABLE

Se determina el salto mediante la diferencia ASCII entre letras guía.

03

VERIFICAR COINCIDENCIA

Se valida si el salto hallado descifra la palabra guía completa correctamente.

04

DESCIFRADO ÚNICO

Se utiliza el único desplazamiento correcto para descifrar el texto una sola vez.

```
// Fragmento Lógico de D&V
for palabra in wordsWithLen: salto = ord(palabra[0]) - ord(palabra_guia[0])
// CALCULAR
if (verificar_coincidencia(...)): desplazamiento_final = salto break
// ¡Clave encontrada!
return descifrar_texto_completo(..., desplazamiento_final)
```

Cifrado

```
# FUNCIÓN cifrar() - Generar texto cifrado
def cifrar():
    global PALABRA_GUIA, FRASE_CIFRADA
    cifrada = []
    # Generar salto_aleatorio (desplazamiento de César)
    salto = random.randint(1, 25)
    fraseMinuscula = FRASE.lower()
    # Seleccionar PALABRA_GUIA con selectGuia()
    PALABRA_GUIA = selectGuia(fraseMinuscula)

    # Recorrer cada carácter de la frase original
    for letra in fraseMinuscula:
        char = ord(letra)
        if char != 32: # Para espacios: copiar tal cual
            # Para letras: aplicar desplazamiento salto_aleatorio
            if char == 241 or char == 209: # Caso especial para 'ñ'
                char = definirChar(110 + salto)
            else:
                char = definirChar(char + salto)
        cifrada.append(chr(char)) # Añadir carácter a lista cifrada

    FRASE_CIFRADA = "".join(cifrada) # Unir lista en FRASE_CIFRADA
```

Seleccionar Guía

```
# Seleccionar palabra guía (mínimo 3 letras)
def selectGuia(frase):
    palabra = ""
    while len(palabra) < 3:
        lista_palabras = frase.split()
        palabra = random.choice(lista_palabras)
    return palabra

# Función auxiliar para manejo de caracteres
def definirChar(ascii):
    if ascii >= 123: # Wrap-around si pasa de 'z'
        return ascii - 122 + 96
    if ascii < 97: # Wrap-around si es menor que 'a'
        return 122 - (96 - ascii)
    return ascii
```


Descifrado

```
def descifradoCesar():  
    wordsWithLen = []  
    for palabra in FRASE_CIFRADA.split():  
        if len(palabra) == len(PALABRA_GUIA):  
            wordsWithLen.append(palabra)  
    # Inicializar desplazamiento en None  
    desplazamiento = None  
    # Para cada palabra en wordsWithLen:  
    for palabra in wordsWithLen:  
        if not palabra or not PALABRA_GUIA:  
            continue  
        # Calcular salto (basado en primera letra)  
        salto = ord(palabra[0]) - ord(PALABRA_GUIA[0])  
        if salto < 0:  
            salto += 26
```

```
    # Asumimos que coincide hasta que se demuestre lo contrario  
    coincidencia_total = True  
    # Iterar cada letra de la palabra guía  
    for i in range(len(PALABRA_GUIA)):  
        letra_cifrada = palabra[i]  
        letra_guia_original = PALABRA_GUIA[i]  
        char_descifrado = definirChar(ord(letra_cifrada) - salto)  
        if chr(char_descifrado) != letra_guia_original:  
            coincidencia_total = False  
            break  
    if coincidencia_total:  
        desplazamiento = salto  
        break  
    # descifrar_after_loop  
    if desplazamiento is None:  
        return "" # No se encontró desplazamiento  
    # Si desplazamiento es diferente a None: descifrar texto completo  
    diccionario = {}  
    for i in range(97, 123): # Crear diccionario de descifrado  
        diccionario[chr(i)] = chr(definirChar(i - desplazamiento))  
    # Aplicar descifrado a toda la frase  
    texto_descifrado = ""  
    for letra in FRASE_CIFRADA:  
        if letra == " ": # Respetar espacios  
            texto_descifrado += " "  
        elif letra in diccionario:  
            texto_descifrado += diccionario[letra]  
        else:  
            texto_descifrado += letra
```


Ejemplo

a, b, c, d, e, f, g, h, i, j, k, l, m, n, ñ, o, p, q, r, s, t, u, v, w, x, y, z

casa -> "c" a "m" hay(10), "a" a "t" hay(21) => **NO COINCIDE**

hola -> "h" a "m" hay(5), "o" a "t" hay(5), "l" a "q" hay(5), "a" a "f" hay(5) => **COINCIDE**

Palabra cifrada: **mtqf**

Resultados en impresión

```
Fuente 1 (Tiempo ns): 56776418454916  
Fuente 2 (Process ID): 3918  
Fuente 3 (Memoria ID): 5061278736  
Semilla caótica aplicada a 'random'.
```

```
Texto cifrado: hq xq oxjdu gh od pdqfkd gh fxbr qrpeuh qr txlhur dfrugduph  
qr kd pxfkr wlhpsr txh ylyld xq klgdojr gh orv gh odqcd hq dvwloohur dgdujd  
dqwljxd urflq iodfr b jdojr fruuhgru
```

```
Palabra guia: rocin
```

```
Desplazamiento encontrado: 3
```

```
Texto descifrado: en un lugar de la mancha de cuyo nombre no quiero  
acordarme no ha mucho tiempo que vivia un hidalgo de los de lanza en  
astillero adarga antigua rocin flaco y galgo corredor
```

Ejecución D&V

```
Fuente 1 (Tiempo ns): 679167495206700
Fuente 2 (Process ID): 21380
Fuente 3 (Memoria ID): 2077841825840
Semilla caótica aplicada a 'random'.
```

COMPARACIÓN PURA DE TIEMPOS DE EJECUCIÓN

```
=====
```

Fuerza Bruta: 25 desplazamientos x todos los caracteres

Divide y Vencerás: Solo desplazamientos probables

```
=====
```

--- Texto: 10 caracteres ---

Fuerza Bruta: 0.000080 segundos, 4126 bytes (memoria real)

Divide y Vencerás: 0.000000 segundos, 2592 bytes (memoria real)

Diferencia memoria: 1534 bytes

--- Texto: 24 caracteres ---

Fuerza Bruta: 0.000180 segundos, 4418 bytes (memoria real)

Divide y Vencerás: 0.000000 segundos, 1912 bytes (memoria real)

Diferencia memoria: 2506 bytes

--- Texto: 32 caracteres ---

Fuerza Bruta: 0.000200 segundos, 4882 bytes (memoria real)

Divide y Vencerás: 0.000000 segundos, 1918 bytes (memoria real)

Diferencia memoria: 2964 bytes

--- Texto: 42 caracteres ---

Fuerza Bruta: 0.000240 segundos, 5542 bytes (memoria real)

Divide y Vencerás: 0.000020 segundos, 1985 bytes (memoria real)

Diferencia memoria: 3557 bytes

--- Texto: 59 caracteres ---

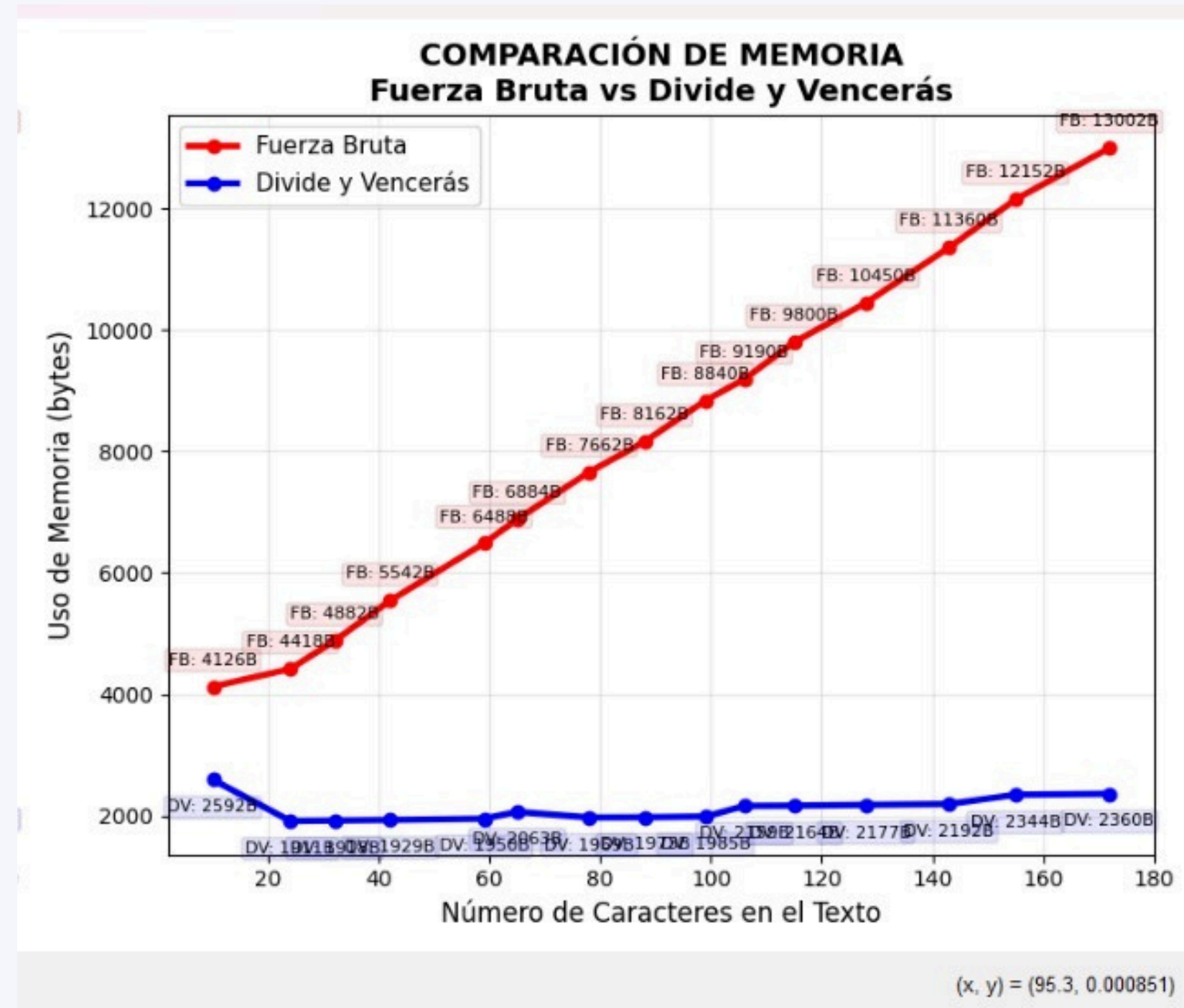
Fuerza Bruta: 0.000340 segundos, 6488 bytes (memoria real)

Divide y Vencerás: 0.000020 segundos, 1946 bytes (memoria real)

Diferencia memoria: 4542 bytes

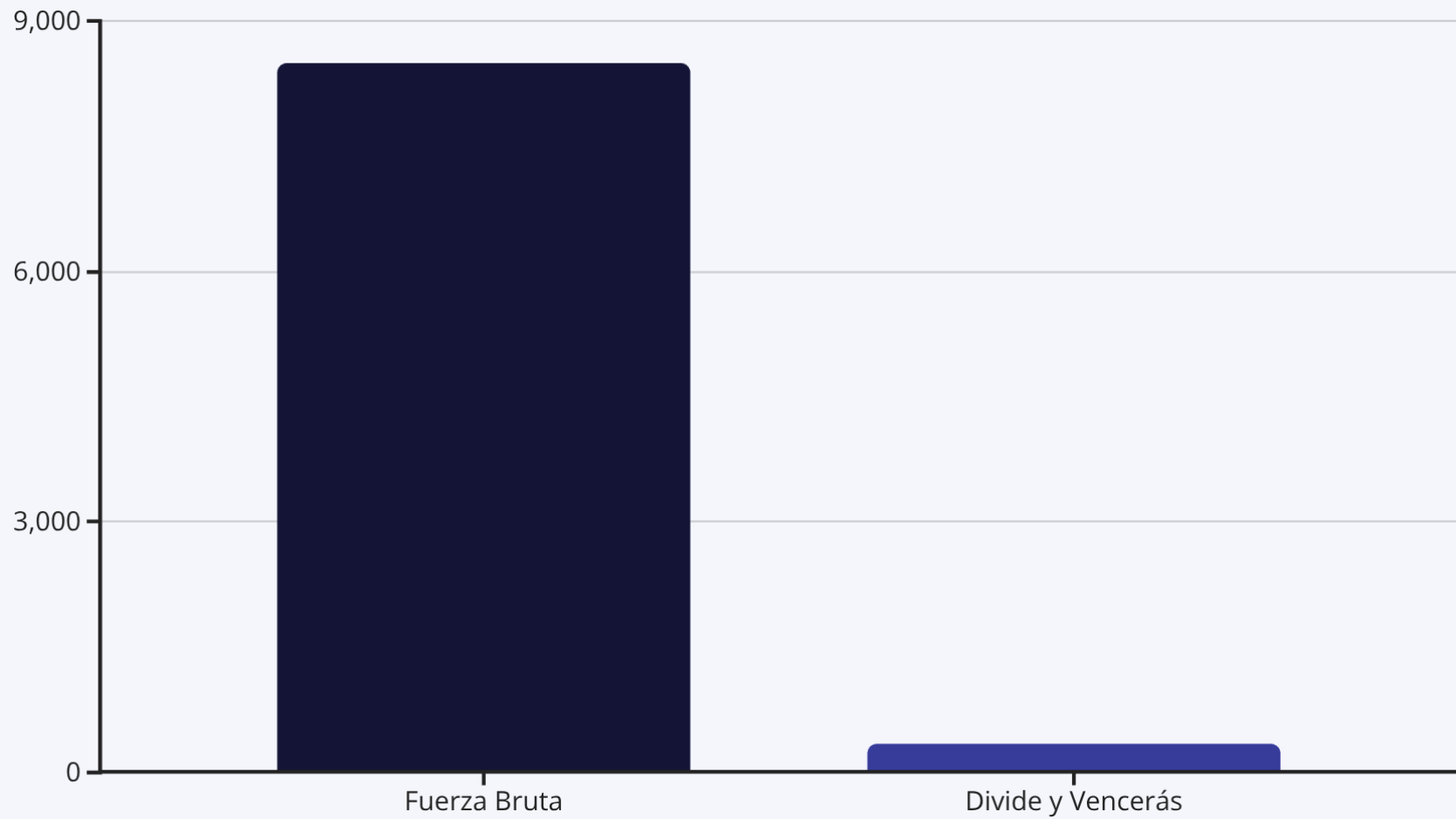
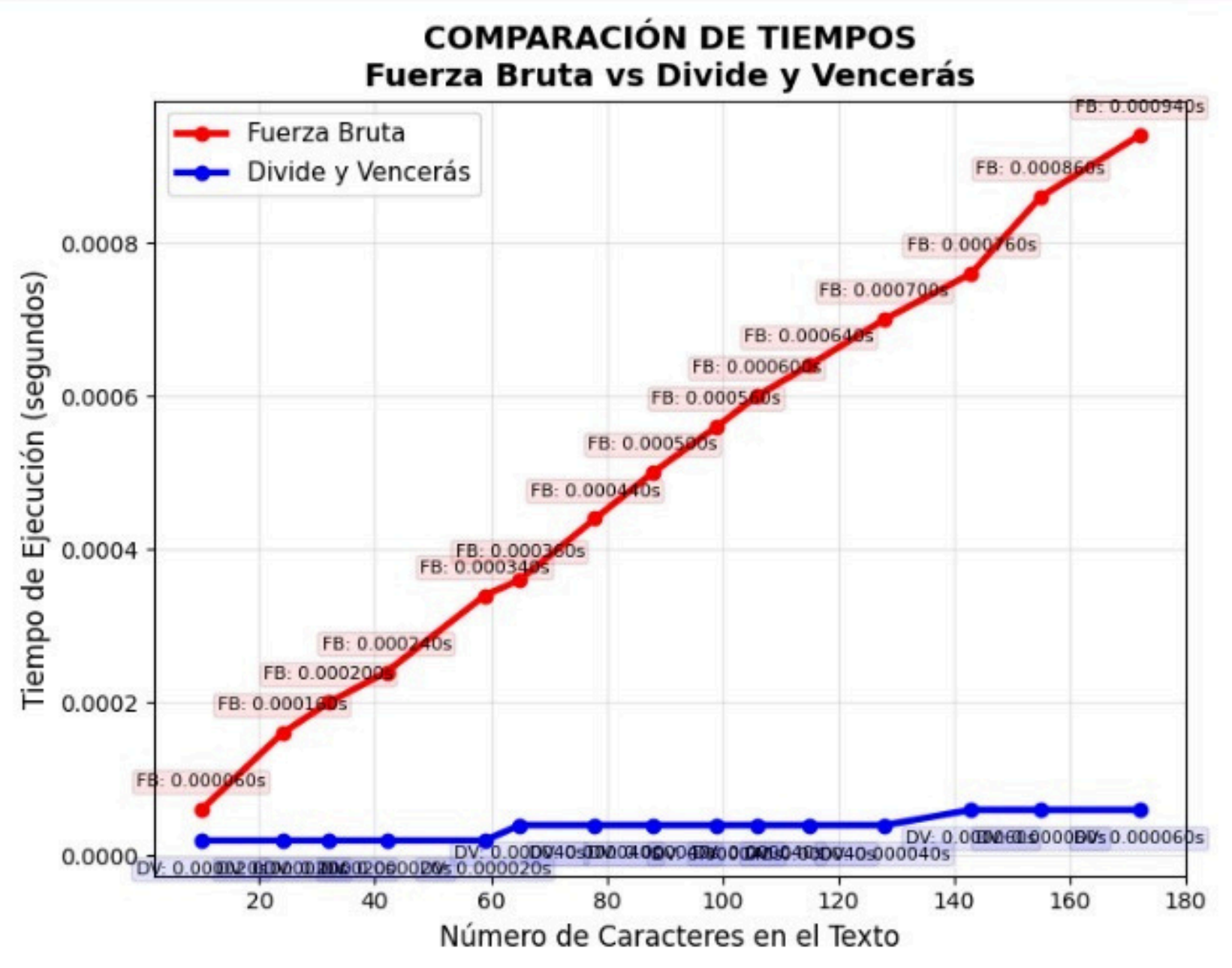
Resultados: tiempo de ejecución

El tiempo de ejecución demuestra la superioridad del enfoque D&V.



- Fuerza Bruta: El tiempo de ejecución escala linealmente conforme el tamaño del texto aumenta.
- Divide y vencerás: Tiempo de ejecución constante y mínimo, con pequeñas variaciones dependiendo del tamaño del texto.

Resultados: Uso de Memoria



- Fuerza Bruta: El uso de memoria escala linealmente con el tamaño del texto (almacena 25 copias).
- Divide y Vencerás: Uso de memoria constante y mínimo, independientemente del tamaño del texto cifrado.
- D&V es 25 veces más eficiente en memoria en promedio.

Conclusiones Clave



SUPERIORIDAD DE D&V

El algoritmo de Divide y Vencerás demostró ser significativamente superior en tiempo y uso de memoria.



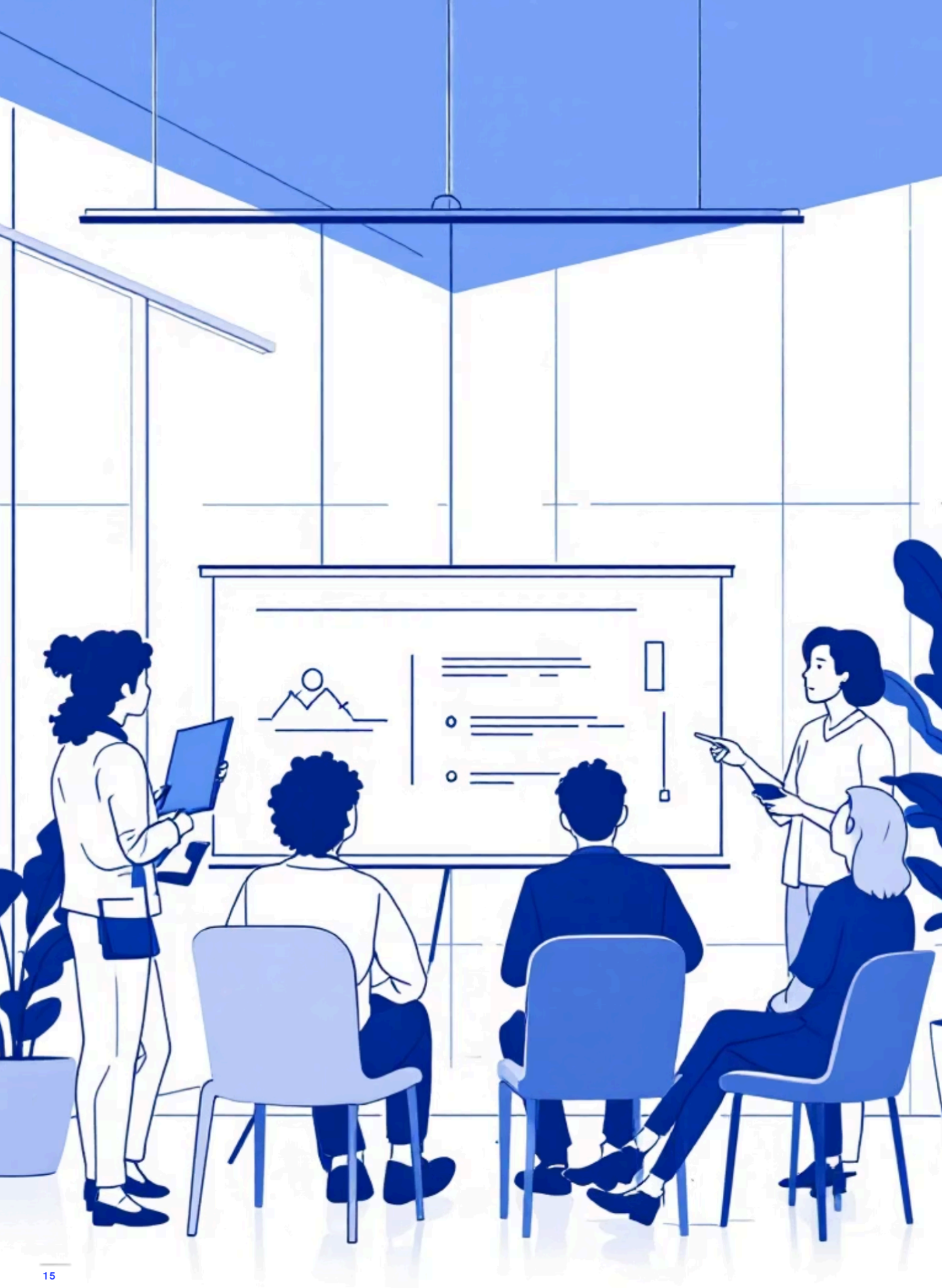
IMPACTO ALGORÍTMICO

La elección del enfoque algorítmico es crítica, incluso para problemas considerados "simples" o de pequeña escala.



OPTIMIZACIÓN POR REDUCCIÓN

Reducir el espacio de búsqueda (de 25 textos a 1 palabra) optimiza drásticamente los recursos computacionales.



GRACIAS POR SU ATENCIÓN

Repositorio del Proyecto: https://github.com/humbertoPena7/Analisis-de-Algoritmos/tree/37dd663da557eb2625506c0d7eca1bf50b427ef1/Equipo_DivideYVenceras